
Site de la mairie d'Angeot

Documentation technique — architecture, logique, sécurité, contenu

Un site institutionnel en PHP natif, sans base de données, sans dépendance et sans étape de construction, conçu pour l'hébergement mutualisé français. Ce document décrit ce que fait le code, pourquoi il le fait ainsi, et ce qu'il refuse de faire.

Dépôt `ftholomier/poubelle` · branche `claudio/mairie-angeot-redesign-t54qiw`
PHP 8.1+ · 63 fichiers PHP · 66 gabarits · 0 dépendance · 0 base de données
Septembre 2026

Sommaire

1. Le parti pris technique

- Ce que le socle refuse, et pourquoi
- Le format des chiffres

2. Structure du dépôt

- L'arborescence
- Les deux implantations possibles
- Les espaces de noms

3. Le cycle d'une requête

- Le contrôleur frontal
- Le routeur
- Les vues
- Les en-têtes, envoyés après le rendu

4. Pas de base de données

- Pourquoi
- Comment le contenu est écrit
- Le verrou optimiste
- Le schéma, les migrations, et l'asymétrie destructrice

5. Le modèle de contenu

- Une page est une suite de blocs typés
- Le catalogue des types
- Quand écrire une vue dédiée
- Les pages, les slugs et le référencement

6. Le back-office

- Authentification
- Les écrans
- La médiathèque et les documents
- La mise à jour depuis le dépôt

7. Sécurité

- Ce qui est exposé, et rien d'autre
- CSRF, sessions, mots de passe
- La politique de sécurité de contenu
- Antispam et quotas
- Échappement et URL
- Permissions de fichiers

8. Vie privée et tiers

- Aucune requête tierce sans consentement
- La mesure d'audience interne

9. Apparence : une teinte, pas une palette

10. Services extérieurs

- L'assistant et le conseiller
- Les réseaux sociaux

Le courrier

Le multilingue

11. La boucle qualité : seize auditeurs

Le principe

Ce que chacun mesure

Les règles qu'ils ont produites

12. Exploitation

Démarrer en local

Mettre en ligne

Ce qui ne se versionne jamais

1. Le parti pris technique

1.1 Ce que le socle refuse, et pourquoi

Ce site est écrit pour un endroit précis : l'hébergement mutualisé français à quelques euros par mois, celui qu'une commune de sept cents habitants peut payer et qu'un secrétariat peut faire durer dix ans. Cet endroit n'offre ni terminal, ni processus long, ni service de base de données fiable, ni chaîne de construction. Chacun des refus ci-dessous découle de là, et aucun n'est une préférence de style.

REFUSÉ	RAISON
Composer, <code>node_modules</code>	Aucune commande à passer sur le serveur. Un dépôt cloné est un site qui marche. Un gestionnaire de paquets, c'est aussi une chaîne d'approvisionnement à surveiller pendant dix ans.
Base de données	Voir § 4. Le contenu d'un site de mairie est de quelques centaines de kilo-octets, lu mille fois plus souvent qu'écrit.
Étape de construction	Le CSS et le JS livrés sont ceux qu'on lit dans le dépôt. Ce qui se déboguait en ligne est ce qui a été écrit.
Cadre applicatif	Le routeur fait 90 lignes, le moteur de gabarits 70. Ce qui manquerait tiendrait en cinquante lignes de PHP.
JavaScript moderne	ES5 tolérant : les postes d'une mairie ne sont pas renouvelés tous les trois ans. Chaque bloc se désarme seul si sa cible est absente de la page.

Deux règles transversales encadrent le tout : **PHP 8.1 minimum**, avec `declare(strict_types=1)` en tête de chacun des 64 fichiers PHP du projet, et **tout est nommé en français** — classes, méthodes, variables, routes, clés JSON. `Antispam::verifier()`, `$valeurs['adresse']`, `/demande-en-ligne`. Le code est relu par des gens qui parlent de « délibérations » et de « démarches », pas de *deliberations* ni de *procedures*.

1.2 Le format des chiffres

63 fichiers PHP · 19 599 lignes	66 gabarits de vue · 8 391 lignes	6 157 lignes de CSS écrites à la main
3 362 lignes de JavaScript	45 fichiers JSON de contenu livré	16 auditeurs · 4 702 lignes

À quoi s'ajoutent 4 004 lignes de documentation en Markdown, versionnées avec le code : `CLAUDE.md` (les consignes de travail), `KIT.md` (la carte du code et les pièges rencontrés), `SITE.md` (les arbitrages propres à Angeot), `NOUVEAU-SITE.md` (la recette pour produire un autre site à l'identique) et `DEPLOIEMENT.md`.

2. Structure du dépôt

2.1 L'arborescence

poubelle/	
├─ public/	seul dossier atteignable par une URL
├─ index.php	contrôleur frontal – le seul point d'entrée
├─ .htaccess	réécriture, en-têtes constants, HSTS
└─ assets/	css/ js/ img/ fonts/ doc/
├─ app/	
├─ Core/	41 classes de service
├─ Admin/	16 contrôleurs et descripteurs du back-office
├─ Controllers/	PageController, ApiController
├─ routes.php	292 lignes – le site public
├─ routes-admin.php	318 lignes – le back-office
└─ helpers.php	e(), t(), lien(), de_nom()...
├─ views/	
├─ layout.php	le gabarit enveloppant
├─ pages/	une vue par page qui en demande une
├─ partials/	fragments, dont bloc.php
└─ admin/	les écrans du back-office
├─ data-modele/	le contenu livré, versionné (45 JSON)
├─ data/	le contenu vivant – JAMAIS versionné
├─ admin/	compte, mot de passe SMTP
├─ pages/	une page = un fichier JSON
└─ assistant/ réseaux/ conseiller/ frequentation/	
├─ storage/	cache, sauvegardes automatiques
├─ config/	amorces, chemins
└─ outils/verifs/	les 16 auditeurs

LA RÈGLE QUI GOUVERNE TOUT LE RESTE

`public/` est le seul dossier exposé. Rien d'autre ne doit être atteignable par une URL. C'est ce qui permet de laisser le compte d'administration, le mot de passe SMTP, les jetons Meta, la clé de l'assistant et les conversations des administrés dans des fichiers en clair sur le disque — et c'est ce que l'auditeur `exposition.php` mesure, dossier par dossier (§ 11.2).

2.2 Les deux implantations possibles

`public/index.php` sait vivre dans deux configurations, et les distingue lui-même :

- **racine web** = `public/` — l'implantation propre, quand l'hébergeur permet de désigner un sous-dossier. Le reste du dépôt est physiquement hors d'atteinte.
- **implantation « à plat »** — le dépôt entier déposé dans `www/`, parce que beaucoup de mutualisés ne savent rien faire d'autre. Là, seuls les `.htaccess` empêchent de télécharger `data/admin/compte.json`. Ils sont donc du code : listés dans `Deployment::CODE`, mis à jour comme le reste, et vérifiés à chaque passage de l'auditeur.

2.3 Les espaces de noms

`App\Core` (41 classes) porte les services : `Content`, `Router`, `View`, `Auth`, `Session`, `Csrf`, `Entetes`, `Antispam`, `Adresse`, `Permissions`, `Verrou`, `Migrations`, `Charte`, `Seo`, `Mediatheque`, `Documents`, `Mailer`, `Assistant`, `Conseiller`, `Connexion`, `Reseaux`, `Publications`, `Diffusion`, `Vignette`,

Deploiement, Frequentation, Langues, Traducteur, TexteRiche, Cookies, Bandeau, Bulle, Avis, Liste, Parametres, Vivant, ConflitEcriture, etc.

App\Admin (16) porte le back-office, un contrôleur par famille d'écrans plus deux descripteurs, Blocs et Contenus. App\Controllers n'en compte que deux : PageController, qui rend les pages publiques, et ApiController, qui répond aux requêtes de l'assistant et des formulaires.

3. Le cycle d'une requête

3.1 Le contrôleur frontal

Toute requête qui ne désigne pas un fichier existant arrive sur `public/index.php`. Il déroule toujours le même enchaînement :

1. **Passe-plat statique** — quand on tourne sous le serveur de développement de PHP, une requête vers un fichier réel est rendue telle quelle.
2. **Amorce** — `$config = require $amorce;` résout les chemins selon l'implantation détectée (§ 2.2).
3. **Construction** — `Content`, `View`, `Router`, puis `require` des deux fichiers de routes.
4. **Dispatch** — le routeur exécute le gestionnaire correspondant, qui rend la page *en mémoire*.
5. **Comptage** — la page vue n'est enregistrée que si la réponse est un HTTP 200, et seulement si `pageVue()` l'accepte : sont écartés `/admin`, `/api/`, `/taches/`, `/assets/`, `/sitemap`, `/robots`, et les agents identifiés comme robots.
6. **En-têtes** — `Entetes::envoyer()`, après le rendu et avant le premier octet écrit (§ 3.4).
7. **Affichage.**

Deux captures encadrent le tout. `ConflitEcriture` — le verrou optimiste (§ 4.3) — provoque une redirection 303 vers `/admin`, en ne reprenant du référent que son chemin, jamais l'hôte. Tout autre `Throwable` part dans le journal d'erreurs du serveur et rend une page d'erreur 500 : rien du message d'origine n'atteint le visiteur.

3.2 Le routeur

`App\Core\Router` tient en une centaine de lignes. Les motifs acceptent des paramètres nommés — `{slug}` — compilés en expression régulière, et deux méthodes seulement sont déclarables : `get()` et `post()`. Deux détails valent d'être notés, parce que chacun a corrigé un défaut réel :

- **HEAD est traité comme GET.** C'est la requête que font les sondes de disponibilité des hébergeurs et les vérificateurs de liens. Sans cela, elles recevaient un 404 : un site déclaré hors service par sa propre supervision.
- **Une adresse connue sous une autre méthode rend 405, pas 404.** Un POST envoyé sur une route déclarée en GET affichait « page introuvable », et l'on cherchait une faute de frappe dans l'URL au lieu d'une méthode de formulaire.

3.3 Les vues

`App\Core\View` rend des gabarits PHP natifs : pas de langage de gabarit, pas de compilation. Une vue est capturée par mise en tampon, puis injectée dans `$slot` à l'intérieur du `layout`. `partial()` inclut un fragment.

UN PIÈGE TROUVÉ PAR UN AUDITEUR

Les variables locales de la méthode de capture sont préfixées (`$_nom`, `$_donnees`, `$_gabarit`), et ce n'est pas une coquetterie. `extract()` en mode `EXTR_SKIP` écarte sans erreur ni alerte toute donnée portant le nom d'une variable déjà présente. Un contrôleur qui passait `'file' => [...]` voyait donc son tableau remplacé par le chemin du gabarit, et la vue parcourait une chaîne de caractères. Trouvé par `alertes.py`, sur le seul indice d'une alerte PHP que personne ne lisait.

3.4 Les en-têtes, envoyés après le rendu

`.htaccess` pose ce qui est constant : `X-Content-Type-Options`, `Referrer-Policy`, HSTS. Trois en-têtes lui échappent, et `App\Core\Entetes` existe pour eux.

Le premier est le cache. Le back-office et l'API reçoivent `Cache-Control: no-store, private` — un mutualisé passe souvent derrière un cache partagé, et une page d'administration retenue là s'affiche au visiteur suivant. Les pages publiques reçoivent `private, no-cache` : le navigateur peut les garder mais doit les revalider. Sans directive du tout, il devinerait une durée et servirait pendant des heures une page que la mairie vient de corriger.

Le deuxième est la politique de sécurité de contenu, détaillée au § 7.3. C'est elle qui impose l'ordre inhabituel du § 3.1 : **la page est assemblée en mémoire, puis les en-têtes sont posés, puis la page est affichée**. La politique ne peut se construire qu'une fois qu'on sait ce que la page contient vraiment.

4. Pas de base de données

4.1 Pourquoi

Le contenu d'un site de mairie tient en quelques centaines de kilo-octets, est écrit quelques fois par mois et lu des milliers de fois. Une base relationnelle apporterait ici : un service à administrer, un mot de passe de plus, une sauvegarde d'un autre genre, un point de panne, une dépendance d'hébergement, un export à négocier le jour où la commune change de prestataire — et rien du côté des besoins réels, qui sont « lire un fichier » et « le réécrire entier ».

Le contenu vit donc en JSON, un fichier par page, dans `data/pages/`. Le contenu de départ est versionné dans `data-modele/` et se recopie tout seul dans `data/` à la première lecture, fichier par fichier : c'est ce qui fait qu'un dépôt fraîchement cloné affiche un site complet sans aucune manipulation, et qu'une mise à jour du modèle ne défait jamais ce que la mairie a écrit.

Une sauvegarde du site, c'est `tar`. Une restauration, c'est `tar`. Un historique, c'est `git` pour le code et un dossier de sauvegardes automatiques pour le contenu.

4.2 Comment le contenu est écrit

`App\Core\Content` expose une API délibérément petite : `load`, `get`, `find`, `publies`, `menu`, `save`, `sauvegardes`, `restaurer`, `modifie`, `traduireEn`.

ÉCRITURES ATOMIQUES — SANS EXCEPTION

Aucun fichier de contenu n'est jamais écrit par un `file_put_contents` direct. On écrit un fichier temporaire, puis `rename()`. Sur un système de fichiers POSIX, `rename()` est atomique : un lecteur voit soit l'ancien fichier entier, soit le nouveau, jamais un JSON tronqué. C'est ce qui remplace ici la transaction d'une base de données, et c'est suffisant parce qu'un fichier est l'unité d'écriture.

La même discipline s'applique aux photos de la médiathèque, aux PDF déposés depuis le back-office (`Documents::televerser()` : envoi vers un nom temporaire, puis `rename()`) et au compteur de quota de l'assistant.

4.3 Le verrou optimiste

L'écriture atomique empêche de lire un fichier à moitié écrit. Elle n'empêche pas la perte silencieuse d'une saisie : la secrétaire ouvre l'éditeur de la page « Déchets », un élu l'ouvre aussi, elle enregistre, il enregistre — son formulaire à lui a été construit avant, il réécrit l'état d'avant, et le travail d'elle disparaît sans qu'aucune erreur ne s'affiche.

`App\Core\Verrou` ne verrouille rien au sens strict : il constate.

- À l'**affichage** d'un écran d'administration (GET), chaque contenu lu laisse son empreinte dans la session de celui qui regarde.
- À l'**enregistrement** (POST), `Content::save()` compare l'empreinte du fichier tel qu'il est maintenant à celle qui avait été relevée. Si elles diffèrent, l'écriture est refusée par une `ConflitEcriture` et l'administrateur est invité à recharger.

Ce découpage GET/POST rend le dispositif invisible : aucun formulaire n'a de champ à porter, aucun contrôleur n'a d'appel à passer. Il tient à une condition unique — **ne jamais relever d'empreinte pendant un POST**, sans quoi la lecture que fait le contrôleur juste avant d'écrire rafraîchirait l'empreinte et le verrou ne verrait plus rien.

Un contenu jamais lu pendant l'affichage n'a pas d'empreinte, et son écriture passe : c'est voulu. Renommer un slug ou une photo réécrit une vingtaine de fichiers que l'écran n'a pas montrés, et refuser ces écritures-là bloquerait le back-office sans rien protéger.

4.4 Le schéma, les migrations, et l'asymétrie destructrice

C'est le point le plus délicat de toute l'architecture, et il n'existe que parce qu'il n'y a pas de base de données pour porter un schéma.

Le schéma vit dans le code — `App\Admin\Blocs::TYPES` — et le contenu vit chez la mairie. La relation entre les deux est **asymétrique** :

- la **lecture est tolérante** : un champ manquant se rend en chaîne vide, et rien ne casse ;
- l'**écriture est destructive** : `Blocs::relire()` reconstruit chaque bloc à partir du schéma courant et jette tout ce que ce schéma ne nomme plus.

Conséquence : renommer un champ ne casse rien tout de suite. Le site s'affiche, tout paraît normal, et la valeur de la mairie disparaît le jour où elle enregistre cet écran — des semaines plus tard, sans message, et sans que personne puisse relier la perte à la livraison.

Deux mécanismes répondent, et ils vont ensemble : **l'un transforme, l'autre constate**.

`App\Core\Migrations` transforme. Chaque fichier de contenu porte un `_version`. À la lecture, les étapes manquantes sont appliquées dans l'ordre et le fichier est réécrit aussitôt : le disque est toujours à la version courante (`VERSION = 2`) et une étape ne se rejoue jamais. Un fichier sans `_version` est réputé être à la version 1. Une étape doit être prudente — ne transformer que ce qu'elle reconnaît avec certitude, laisser le reste intact : une migration qui écrase du texte écrit par la mairie est pire que l'absence de migration.

`outils/verifs/schema.php` constate. Une migration ne s'écrit pas d'elle-même : renommer un champ en oubliant l'étape laisse le mécanisme muet. Ce script confronte tout le contenu au schéma et nomme le bloc, le champ et la conséquence. Deux secondes, aucun navigateur.

5. Le modèle de contenu

5.1 Une page est une suite de blocs typés

Il n'y a pas de vue par page. Une page est un tableau JSON de blocs, chacun portant un `type` et ses valeurs. Les types sont décrits *une seule fois* dans `app/Admin/Blocs.php` et rendus par `views/partials/bloc.php`.

Cette description unique sert simultanément trois choses : l'écran d'édition du back-office (un seul formulaire les édite tous), la relecture du POST, et le rendu public. Ajouter une page ne coûte donc qu'un fichier JSON, une entrée dans `Seo::PAGES` et une ligne de route.

Chaque champ porte une **nature**, qui dit à la fois comment il se saisit et comment il se relit :

NATURE	SAISIE ET STOCKAGE
ligne / zone	une ligne de texte ; un petit paragraphe
riche	texte long mis en forme — gras, listes, bouton — filtré par <code>App\Core\TexteRiche</code> et stocké en HTML
paragraphe	texte long en clair, découpé aux lignes vides
lignes	une liste, une entrée par ligne
photo	un choix dans la médiathèque
fichier	chemin d'un PDF, relatif à <code>public/</code>
lien	couple libellé + adresse, nettoyée par <code>Adresse</code>
ancree	l'identifiant qui permet de pointer la section
fond	blanc, teinté, sombre, ou l'alternance automatique
icone	un choix parmi les 27 pictogrammes du site
date	un jour, saisi au calendrier du navigateur, stocké en ISO
choix:a b	une liste fermée
items:<clé>	une liste d'entrées, décrite par <code>SOUS_BLOCS</code>

5.2 Le catalogue des types

Treize types de blocs : `texte`, `duo`, `cartes`, `liens`, `contacts`, `documents`, `etapes`, `tableau`, `encadre`, `hebergeur`, `citation`, `chiffres`, `photo`. Huit d'entre eux contiennent des listes d'entrées décrites par `SOUS_BLOCS` — les cartes d'une grille, les lignes d'un tableau, les étapes d'une démarche.

5.3 Quand écrire une vue dédiée

La règle est explicite, et elle est restrictive : **on n'écrit une vue dédiée que si la mise en page dépend d'une donnée structurée** — le trombinoscope du conseil municipal, la liste filtrable des démarches. Jamais pour du texte suivi. Une vue dédiée écrite par confort, c'est une page que la mairie ne peut plus modifier seule.

5.4 Les pages, les slugs et le référencement

`App\Core\Seo::PAGES` est le registre unique : trente-cinq entrées, chacune associant une clé interne à un slug et un nom affiché. Ce registre alimente à lui seul les routes, le fil d'Ariane, le plan du site, le `sitemap.xml` et les

redirections 301 depuis les adresses de l'ancien site. Une clé inconnue passée à `Seo::page()` est journalisée plutôt qu'ignorée en silence.

6. Le back-office

6.1 Authentification

Il n'y a **aucun identifiant par défaut**. Le compte vit dans `data/admin/compte.json`, hors dépôt git et hors racine web ; tant qu'il n'existe pas, `/admin` propose l'écran de première configuration. Le mot de passe est haché par `password_hash()` avec l'algorithme par défaut de PHP, et réhaché automatiquement si `password_needs_rehash()` le demande — c'est ce qui fait qu'un compte créé aujourd'hui suivra le durcissement des versions futures de PHP sans intervention.

BORNE	VALEUR	RAISON
Échecs consécutifs	5	par IP, sur une fenêtre d'une heure
Verrouillage	15 min	rend la force brute inopérante sans bloquer un élu qui se trompe
Inactivité	2 h	le poste du secrétariat est dans une pièce où passent des administrés, et le navigateur y reste ouvert d'un jour sur l'autre

La durée d'inactivité est publique (`Auth::INACTIVITE_SEC`) parce que `Session` doit y aligner `session.gc_maxlifetime` : deux durées séparées, c'est la plus courte qui gagne, sans que personne comprenne pourquoi la déconnexion arrive avant l'heure annoncée. La session porte son propre nom (`mairie_session`) et `session.use_strict_mode` est activé, ce qui interdit qu'un identifiant de session soit imposé de l'extérieur.

6.2 Les écrans

Seize contrôleurs, un par famille : contenu et édition de blocs, apparence, paramètres, médiathèque, référencement, avis et démarches, langues, réseaux sociaux, conversations de l'assistant, conseiller éditorial, mise à jour. Chaque formulaire porte `Csrf::champ()` et chaque traitement commence par `Csrf::verifier()` — sans exception.

6.3 La médiathèque et les documents

Les photos passent par `Mediatheque`, qui redimensionne et produit les vignettes. Les PDF passent par `App\Core\Documents`, ajouté parce que l'aide disait auparavant « les PDF se déposent par FTP » : publier un compte-rendu de conseil est la seule tâche vraiment mensuelle d'un secrétariat, et elle demandait une adresse de serveur et un mot de passe que personne n'a sous la main un mardi matin. La conséquence n'était pas un inconfort, c'était une rubrique qui cesse d'être tenue à jour.

Trois garde-fous, dans cet ordre :

1. **La signature du fichier**, pas son extension ni son type déclaré. Un navigateur annonce ce qu'on lui dit d'annoncer et `.pdf` n'est qu'un suffixe : seuls les cinq premiers octets, `%PDF-`, font foi. Plafond : 15 Mio.
2. **Le nom est réécrit, jamais repris**. Un nom arrive d'un poste Windows avec des espaces, des accents et parfois des points en série — et c'est aussi par là que passe un `../`. Un homonyme reçoit un suffixe numérique plutôt que d'écraser.
3. **L'écriture est atomique**. Un envoi interrompu laisserait sinon un PDF tronqué, publié et illisible.

Le dossier reçoit en outre son propre `.htaccess` : il vit sous `public/`, donc servi, et reçoit désormais des fichiers venus d'un navigateur. Il coupe l'exécution (`php_flag engine off`, `RemoveHandler`, `Options -ExecCGI`) et

force `Content-Disposition: attachment` — un PDF affiché en ligne s'exécute dans l'origine du site, et un PDF sait porter du JavaScript.

6.4 La mise à jour depuis le dépôt

`App\Core\Deploiement` met le site à jour depuis git, depuis le back-office, sans terminal. Le principe de sûreté tient en une phrase : **la mise à jour ne touche QUE les chemins listés dans `CODE` . `data/`**, les photos envoyées et les sauvegardes n'y sont jamais, donc ne sont jamais écrasés — même si le dépôt en contient une version différente.

La liste couvre `app/`, `config/`, `views/`, `public/index.php`, les feuilles de style et scripts, les logos, `data-modele/`, et — explicitement — les cinq `.htaccess` de sécurité, qui sont du code et non du contenu. Avant chaque mise à jour, `data/` et les photos du site sont sauvegardés dans une archive. L'appel à git passe par `proc_open` sans shell : aucun argument ne traverse un interpréteur de commandes.

7. Sécurité

7.1 Ce qui est exposé, et rien d'autre

Dans l'implantation à plat (§ 2.2), tout le dépôt est sous la racine web et seuls les `.htaccess` retiennent les secrets. Or trois d'entre eux étaient déclarés dans `Deploiement::CODE`, vantés par la documentation — et n'existaient pas. Le compte d'administration, le mot de passe SMTP, les jetons Meta, la clé de l'assistant et les conversations des administrés étaient téléchargeables. Rien ne le montrait : **un refus absent ne produit aucune erreur**.

D'où `outils/verifs/exposition.php`, qui mesure le système de fichiers et non une page. La règle qu'il applique vaut mieux qu'une liste : **tout dossier du dépôt sauf `public/` doit refuser l'accès** — ce qui couvre d'avance celui qu'on ajoutera demain. Il vérifie aussi que chaque `.htaccess` déclaré dans `Deploiement::CODE` existe réellement, et que chaque dossier listé par `Permissions::secrets()` est couvert.

7.2 CSRF, sessions, mots de passe

Un jeton par session, posé par `Csrf::champ()` dans chaque formulaire d'administration et vérifié par `Csrf::verifier()` en tête de chaque traitement. Un échec ne se solde pas par un silence : il affiche un message expliquant qu'une session a expiré et ce qu'il faut faire — parce qu'un rejet muet après vingt minutes de rédaction est perçu comme un bogue, et fait recommencer.

Côté session : nom propre, `use_strict_mode`, `gc_maxlifetime` aligné sur l'inactivité, régénération de l'identifiant à la connexion.

7.3 La politique de sécurité de contenu

La CSP porte un **nonce tiré au sort à chaque requête** (seize octets, soit les 128 bits recommandés). Un fichier de configuration Apache ne peut pas le connaître ; sans nonce, il faudrait `'unsafe-inline'` sur les scripts, ce qui revient à ne pas avoir de CSP du tout puisque c'est précisément le script injecté que la politique doit arrêter.

Surtout, **ce qui est autorisé dépend de la page**. Le plan d'accès est une `iframe` dont l'adresse est collée par la mairie : elle peut venir de Google, d'OpenStreetMap ou d'ailleurs. Une liste écrite à la main se périmerait au premier changement et le plan disparaîtrait sans message ni erreur. Les fragments qui chargent un tiers le déclarent donc eux-mêmes — `Entetes::autoriserCadre()` pour une `iframe`, `autoriserProtecteur()` pour le test anti-robot — et la politique se construit à la fin du rendu, quand on sait ce que la page contient vraiment.

Résultat : une page sans formulaire ne nomme pas Cloudflare, une page sans plan ne nomme personne, et un hôte tiers n'est accordé que si un réglage l'a réellement allumé. Sans identifiant de mesure, Google n'y figure pas.

7.4 Antispam et quotas

`App\Core\Antispam` combine quatre barrières, dans l'ordre du moins au plus coûteux : un champ piège invisible, une horloge signée en HMAC (un formulaire renvoyé en deux secondes n'a pas été rempli par un humain), un quota horaire par empreinte d'IP hachée, et — en option — le test Turnstile de Cloudflare, qui n'est chargé que si la clé est renseignée.

Le quota est découpé en **familles** — formulaire de contact, assistant, rappel téléphonique — chacune avec son propre plafond, pour qu'un usage intensif de l'un ne ferme pas les autres.

UNE DÉPENSE QUI NE SE VOYAIT NULLE PART

Le quota de l'assistant vivait dans `$_SESSION` : un script sans cookie repartait de zéro à chaque appel, et la facture Gemini de la mairie n'avait plus de borne. Aucune page ne le montre — le site répond normalement, et le défaut ne se voit que sur un relevé Google. Le compteur est désormais journalier et sur disque (`Assistant::PLAFOND_JOUR = 400`), et `outils/verifs/quota.php` le mesure avec une doublure qui tient lieu de Gemini : aucune requête ne sort, rien n'est facturé, et c'est le vrai code de l'API qui est exercé.

Quand l'assistant refuse, il refuse en donnant d'abord les numéros d'urgence. C'est une règle du projet, pas un détail d'ergonomie : **les urgences passent avant tout le reste**, y compris avant un renvoi vers une page du site.

7.5 Échappement et URL

`e()` sur toute sortie, sans exception — c'est la règle la plus mécanique du dépôt, et la plus facile à vérifier en relecture.

Les adresses passent par `App\Core\Adresse`, **un validateur unique appliqué deux fois : à l'écriture et au rendu**. Valider seulement à l'écriture laisserait passer tout ce qui est déjà en base ; valider seulement au rendu laisserait la mairie croire son lien enregistré. Le nettoyage retire les caractères de contrôle (`\0 \t \n \r \v \f`) mais non les espaces avant de détecter le schéma — un `java\tscript:` passait sinon —, n'accepte que `http` et `https` (plus `mailto:` et `tel:` pour les champs de contact), rejette les adresses protocole-relatives `//hôte`, et rend une chaîne vide dans tous les autres cas.

7.6 Permissions de fichiers

`App\Core\Permissions` centralise les modes et la liste des dossiers « secrets » : `data/admin`, `data/assistant`, `data/reseaux`, `data/conseiller`. Un `.htaccess` n'est jamais traité comme un secret — Apache doit pouvoir le lire — et c'est exactement le genre de détail qui, oublié, désarme silencieusement toute la protection.

8. Vie privée et tiers

8.1 Aucune requête tierce sans consentement

C'est vrai partout, mais une mairie n'a pas le droit de déposer les cookies d'un autre avant que l'administré ait dit oui. Le site ne charge donc, tant qu'aucun accord n'a été donné, **aucune ressource extérieure** : polices, scripts, images et cartes viennent tous du domaine. L'auditeur `traceurs.py` le mesure page par page et sort en erreur au premier hôte tiers.

8.2 La mesure d'audience interne

`App\Core\Frequentation` répond à la question « la page Déchets est-elle lue ? » sans rien envoyer nulle part. Le socle sait brancher Google Analytics, mais celui-ci ne se charge qu'après accord : sur un site de mairie, la moitié des gens refusent, et le chiffre obtenu ne dit plus rien de la fréquentation réelle.

Ce qui est enregistré, et c'est tout : une date, un chemin, un nombre. Pas d'adresse IP, pas de cookie, pas d'identifiant, pas d'agent utilisateur, pas de référent, pas d'horaire à la seconde. Rien qui permette de suivre quelqu'un d'une page à l'autre ni d'un jour au lendemain. C'est ce qui range cette mesure dans l'exemption de consentement prévue par la CNIL pour la mesure d'audience strictement nécessaire — et c'est aussi pourquoi elle ne peut pas dire « combien de visiteurs », seulement « combien de pages vues ». La nuance est écrite à l'écran plutôt que maquillée.

Un fichier par mois, une ligne par jour ; treize mois gardés, de quoi comparer un mois à celui de l'an dernier ; au-delà de 400 pages distinctes, le détail par page cesse pour qu'un mois ne puisse pas enfler.

9. Apparence : une teinte, pas une palette

Le projet pose une règle générale : *tout réglage laissé à la mairie doit avoir son auditeur qui en force les bornes*. Un sélecteur de couleur libre est le pire des réglages de ce point de vue — il touche les cent cinquante mesures de contraste du site d'un seul clic. Une couleur posée telle quelle dans `--bleu` rendrait invisible la moitié des libellés dès que quelqu'un choisirait un jaune ou un rose pâle.

D'où le parti pris de `App\Core\Charte` : **la mairie choisit une teinte, pas une palette**. Les cinq tons de marque et les neutres teintés en sont dérivés :

- la **teinte** vient du choix et se propage à tout, marque et neutres ;
- la **saturation** vient du choix pour les tons de marque, bornée à [18 %, 55 %] — sous 18 % le site paraît en panne, au-delà de 55 % les aplats deviennent criards sur un site de service public. Les neutres gardent la leur, faible et fixe ;
- la **luminosité** des tons de marque n'est pas choisie : elle est **résolue**, ton par ton, jusqu'à atteindre le contraste que ce ton doit tenir sur le fond où il sert.

C'est ce troisième point qui fait tout le travail. Le ton qui porte les petits libellés sur le crème est assombri jusqu'à 6:1 sur ce crème-là, quelle que soit la teinte ; celui des accents sur fond sombre est éclairci jusqu'à 4,6:1 sur la plus claire des surfaces sombres du site. **La mairie ne peut donc pas produire une page illisible, et n'a rien à savoir du contraste**. La teinte livrée pour Angeot est le bleu ardoise du blason, `#456d8a`.

`outils/verifs/couleur.py` force ce réglage sur douze teintes réparties sur la roue plus quatre cas limites — gris sans saturation, rouge saturé, presque noir, presque blanc — et repasse les auditeurs de contraste. C'est lui qui *prouve* la propriété, plutôt que de la promettre.

10. Services extérieurs

10.1 L'assistant et le conseiller

Deux usages de Gemini, tous deux éteints tant qu'aucune clé n'est renseignée. **Assistant** répond aux administrés depuis une bulle du site public ; **Conseiller** aide la secrétaire à rédiger depuis le back-office. **Connexion** porte l'appel HTTP commun.

Le corpus est **reconstruit à chaque question** à partir du contenu réel du site : l'assistant ne peut donc pas répondre sur une version périmée d'une démarche, et une correction faite le matin est prise en compte l'après-midi. Sa consigne lui impose de donner les numéros d'urgence avant tout autre chose, et de ne jamais inventer une règle administrative.

10.2 Les réseaux sociaux

Reseaux gère la connexion OAuth aux comptes Meta depuis le back-office, **Publications** tient la file d'attente, **Diffusion** publie, et **Vignette** fabrique avec GD une image carrée quand une actualité n'a pas de photo. Une tâche planifiée appelle `/taches/reseaux?cle=...`.

UNE BRANCHE QU'ON NE PEUT ATTEINDRE QU'EN PRODUCTION

Ce que fait la file quand Facebook accepte et qu'Instagram refuse ne s'affiche nulle part : il faut publier vraiment pour le voir, donc personne ne le voit jamais. La branche était fausse — la publication était retirée de la file et inscrite au journal comme réussie, Instagram n'était jamais retenté, et la mairie croyait avoir publié partout. `utils/verifs/file.php` substitue une doublure à Meta : aucune requête ne sort, et vingt-huit mesures tiennent en deux secondes. **Une branche qu'on ne peut atteindre qu'en production doit avoir sa doublure, sans quoi elle n'est jamais exercée.**

10.3 Le courrier

Mailer envoie par SMTP, avec les identifiants stockés dans `data/admin/`. Aucune bibliothèque : le dialogue SMTP est écrit à la main, ce qui suffit largement pour un accusé de réception de formulaire.

10.4 Le multilingue

Les textes d'interface passent tous par `t('...')` et sont relevés automatiquement pour la traduction — il n'y a rien à déclarer. **Langues** gère les langues actives, **Traducteur** les dictionnaires, **TraductionAuto** la traduction assistée du contenu, et la langue se porte par un préfixe d'URL.

11. La boucle qualité : seize auditeurs

11.1 Le principe

Le niveau de ce projet ne vient pas du goût, mais de la mesure. Seize scripts sont à faire passer avant de déclarer une tâche finie. **Chacun sort en code 1 s'il trouve quelque chose ; zéro écart, zéro souci, zéro hôte tiers, c'est la définition de « fini » ici, pas une option.**

Douze pilotent un navigateur (Playwright/Chromium) ; quatre ne regardent aucune page. Ils ne doivent **pas être lancés en parallèle sans y penser** : plusieurs écrivent dans `data/` pour forcer un réglage — `couleur.py` la teinte, `bulle.py` l'assistant — et deux qui s'y croisent se mesurent l'un l'autre.

11.2 Ce que chacun mesure

AUDITEUR	CE QU'IL MESURE
<code>contraste.py</code>	le contraste réel de chaque texte, tel que rendu
<code>mise-en-page.py</code>	débordement, cibles tactiles, hiérarchie des titres, textes alternatifs, superpositions — à cinq largeurs, téléphone compris
<code>mise-en-page.py --admin</code>	les mêmes, sur le back-office
<code>traceurs.py</code>	aucune requête tierce sans accord
<code>bandeau.py</code>	chaque photo du diaporama, voile forcé à sa borne basse
<code>entete.py</code>	l'en-tête aux deux bornes du réglage de logo, deux modes, deux dispositions — 96 mesures
<code>couleur.py</code>	le site sous seize teintes de commune
<code>bulle.py</code>	le bouton de l'assistant sous 251 réglages
<code>conseiller.py --admin</code>	la pastille du conseiller sous ses trois états, réponses du modèle doublées
<code>vignette.py</code>	l'image carrée fabriquée pour Instagram, pixel par pixel, sous chaque couleur
<code>alertes.py</code>	ce que PHP dit tout bas, et que la page ne montre pas
<code>file.php</code>	la file de publication quand un seul réseau répond
<code>schema.php</code>	le contenu écrit et le schéma du code disent-ils la même chose
<code>exposition.php</code>	ce qu'une URL atteindrait si la racine web était celle du dépôt
<code>quota.php</code>	ce que l'assistant refuse, et à qui
<code>aller-retour.py</code>	enregistrer sans rien changer : le JSON ne doit pas maigrir

11.3 Les règles qu'ils ont produites

Chaque auditeur existe parce qu'un défaut est passé. Les règles qui en découlent valent mieux que la liste des scripts, parce qu'elles s'appliquent à ce qu'on écrira demain.

Ce qui ne se voit pas dans la page doit être mesuré là où il se voit.

`alertes.py` est le plus utile des seize : il lit le *journal d'erreurs de PHP*, que personne ne lisait. Une alerte ne sort pas dans la page, puisque `display_errors` est éteint en production — et doit l'être. La vérification que le socle proposait, `curl ... | grep -ci "warning"`, ne mesurait donc que ce qui ne devrait jamais arriver. Le jour où ce script a existé, il a trouvé en trois minutes trois défauts silencieux : deux sections de contenu qui ne s'affichaient pas du tout, et une donnée passée à un gabarit qui n'y arrivait jamais.

Un réglage qui décide de la PRÉSENCE d'un élément cache cet élément aux auditeurs.

L'assistant est éteint tant qu'aucune clé n'est renseignée : sa bulle n'est donc pas dans la page que les autres mesurent. C'est ce trou qui a laissé passer, pendant toute la vie du socle, un libellé à 2,57:1 — l'encre sur la couleur de marque. Il faut alors un script qui allume l'élément pour le mesurer.

Quand ce qu'il faut mesurer n'apparaît qu'après la réponse d'un service extérieur, on double le service.

Allumer la pastille du conseiller ne suffisait pas : le panneau ouvert est *vide*. Les bulles de conversation et les fiches du bilan n'existent qu'*après* une réponse de Google. Un auditeur qui se contentait d'ouvrir le panneau mesurait un cadre et deux onglets, et déclarait le tout conforme. On ne saute pas la mesure : on double le service.

Quand une interface a des états, mesurez d'abord qu'elle est dans l'état où vous croyez la mesurer.

C'est la leçon la plus utile des seize. `conseiller.py` mesurait des tailles, des contrastes et des débordements — tout au vert — pendant que le panneau était en réalité ouvert avant tout clic et que les deux vues s'empilaient. Cause : une règle `display: flex` explicite l'emporte sur l'attribut `hidden`, dont la spécificité est nulle. Le JS basculait `hidden`, le CSS l'ignorait, changer d'onglet ne faisait rien de visible — et c'est l'utilisateur qui l'a signalé, pas l'auditeur. **Un auditeur qui ne mesure que la géométrie mesure juste sur une page fausse.** Il vérifie désormais l'état avant tout le reste.

Tout réglage laissé à la mairie doit avoir son auditeur qui en force les bornes.

Un curseur dont on n'a mesuré que la valeur livrée est un défaut en attente, découvert en ligne par la mairie. C'est ce qui a rattrapé le voile du bandeau : réglable de 0 à 100 et mesuré à sa seule valeur du jour, il laissait servir un titre blanc sur une photo claire.

Le back-office se mesure aussi.

Il ne l'était pas, et c'est là qu'un défaut a vécu tranquillement : une pastille de code faite pour occuper sa propre ligne, posée au fil d'un paragraphe, recouvrait la ligne du dessus sur deux écrans — 46 px de haut dans une ligne de 22. Le rembourrage vertical d'une boîte *en ligne* ne pousse pas la ligne : il déborde. Le même passage a relevé une centaine de cibles tactiles sous 44 px jusque-là non mesurées.

Les quatre causes de débordement latéral sur téléphone.

Le passage du back-office à 320 px a demandé de lever quatre causes, toutes classiques et toutes invisibles sur un grand écran. La liste couvre à peu près tout ce qui fait défiler une page latéralement :

- `min-width: 0` sur ce qui doit pouvoir rétrécir. Un enfant de flex ou de grille refuse de descendre sous la largeur minimale de son contenu tant qu'on ne le lui dit pas. De loin la cause la plus fréquente.
- `minmax(min(190px, 100%), 1fr)` et jamais `minmax(190px, 1fr)` : une piste de grille garde son minimum même quand la fenêtre est plus étroite qu'elle, et pousse la page dehors.
- `fieldset { min-inline-size: 0 }` : la feuille du navigateur y met `min-content`, qu'aucune largeur ne surcharge. Bizarrerie historique du HTML, pas un choix du socle.
- Une grille à colonnes fixes ne se resserre pas : elle doit devenir une rangée qui se replie.

LA RÈGLE DE CONDUITE

N'affaiblissez jamais un auditeur pour faire passer une page. Si un résultat semble faux, c'est probablement un vrai faux positif ; corrigez le script en expliquant pourquoi, ne relevez pas le seuil. Un piège de mesure connu : `run ( ) { "$@" | tail -4; }` rapporte le code de sortie de `tail`, pas celui de l'auditeur — un script qui imprime « 6 souci(s) » au-dessus d'un « [code 0] » a déjà trompé une relecture.

Ce que ces scripts ont attrapé et qu'aucun œil n'avait vu : un texte à 4,36:1 sur le fond crème, la couleur de marque à 2,16:1 sur l'anthracite, un sur-titre à 2,04:1 sur une photo de bandeau, des cibles tactiles à 26 px, un saut de titre h1 → h3 sur une fiche de démarche, un logo servi écrasé au-delà de 100 px, et le blanc à 90 % d'opacité du chapô — 4,05:1 là où le blanc plein donne 4,9:1.

12. Exploitation

12.1 Démarrer en local

```
php -S localhost:8080 -t public public/index.php
```

`data-modele/` se recopie seul dans `data/` : le site s'affiche complet sans manipulation. `/admin` crée le compte administrateur au premier passage — il n'y a aucun identifiant par défaut.

Vérifications rapides pendant le travail :

```
php -l views/pages/accueil.php # syntaxe
curl -s localhost:8080/ | grep -ci "warning\\|fatal\\|notice" # doit rendre 0
```

Et pour le back-office, la seule qui compte vraiment : **ouvrir un écran ne suffit pas, ce qui casse c'est l'enregistrement**. Un champ mal nommé vide une clé sans rien signaler. Après toute modification d'un écran d'édition, enregistrez sans rien changer et comparez le JSON avant/après : il ne doit pas maigrir. `aller-retour.py` le fait tout seul, sur les écrans déclarés en tête du script — **ajoutez-y tout écran d'édition que vous créez**, un écran absent de sa liste n'est pas vérifié, et c'est exactement ainsi que le défaut revient.

12.2 Mettre en ligne

Deux voies, décrites par `DEPLOIEMENT.md` : un dépôt cloné sur le serveur puis mis à jour depuis le back-office (§ 6.4), ou un dépôt de fichiers si l'hébergeur n'offre rien d'autre. Dans les deux cas, la racine web doit pointer sur `public/` quand c'est possible, et les `.htaccess` doivent être présents quand ça ne l'est pas.

12.3 Ce qui ne se versionne jamais

CHEMIN	CONTENU
<code>data/admin/</code>	compte administrateur, mot de passe SMTP
<code>data/*.json</code> , <code>data/pages/</code>	le contenu vivant écrit par la mairie
<code>data/frequentation/</code>	les pages vues
<code>storage/</code>	cache et sauvegardes automatiques

Seul `data-modele/` est versionné : c'est la source, et c'est lui qui doit le rester. Après un essai du back-office en local, un aller-retour dans un formulaire laisse du contenu modifié dans `data/` ; on remet donc à zéro avant de committer :

```
rm -rf data/pages data/*.json data/admin \
    storage/cache/* storage/sauvegardes/*
```

CE QU'UN SITE DE SERVICE PUBLIC EXIGE EN PLUS

Ces règles ne s'appliquaient pas au site commercial dont vient le socle. Elles s'appliquent ici, et une modification qui les casse est un défaut.

Ne publiez jamais une règle administrative que vous n'avez pas vérifiée. Une démarche change de pièces, de seuils et de délais d'une année à l'autre ; une fiche périmée envoie un administré au guichet avec le mauvais dossier. Le site précédent décrivait l'urbanisme d'avant 2012 et les procurations d'avant 2022.

N'affirmez jamais un service que la mairie ne rend pas. La fiche « Carte d'identité » dit que la commune n'a pas de dispositif de recueil, parce que c'est vrai d'une commune de sept cents habitants ; le texte d'origine, recopié d'un modèle, affirmait le contraire.

Les urgences passent avant tout le reste, y compris avant un renvoi vers une page du site.

Aucune requête tierce sans consentement.